

Berliner Hochschule für Technik

Bachelorarbeit

Titel der Arbeit

Vorname Nachname

Matrikelnummer: 123456

Studiengang: Medieninformatik

Betreuung: Prof. Dr. Name

Abgabedatum: TT.MM.JJJJ

Vorwort

Diese Bachelorarbeit entstand im Studiengang Medieninformatik an der Berliner Hochschule fuer Technik. Sie behandelt die Konzeption und Implementierung einer interaktiven Anwendung zur Veranschaulichung praemptiver CPU-Scheduling-Verfahren.

Das Vorwort wird in der Endfassung bei Bedarf um persoenliche Danksagungen und organisatorische Hinweise ergaenzt.

Contents

Vorwort	i
List of Figures	iv
List of Tables	v
Abkuerzungsverzeichnis	vi
1 Einleitung	1
2 Grundlagen	2
3 Didaktische Grundlagen interaktiver Visualisierungen	3
3.1 Lernen mit digitalen Medien	3
3.2 Multimediales und visuelles Lernen	3
3.3 Didaktischer Nutzen von Simulationen	3
3.4 Visualisierung komplexer Ablaeufe im Lernkontext	4
3.5 Anforderungen an die didaktische Aufbereitung von Scheduling-Prozessen	4
4 Grundlagen der Mensch-Computer-Interaktion	6
4.1 Ziele der Mensch-Computer-Interaktion	6
4.2 Usability und Nutzerfreundlichkeit	6
4.3 Interaktionsgestaltung fuer Lernanwendungen	6
4.4 Rueckmeldung, Transparenz und Verstaendlichkeit	7
4.5 Anforderungen an die Benutzungsoberflaeche des Visualizers	7
5 Anforderungen und Konzeption des Scheduling-Visualizers	9
5.1 Zielgruppe und Einsatzkontext	9
5.2 Fachliche Anforderungen	9
5.3 Didaktische Anforderungen	10
5.4 Funktionale Anforderungen	10
5.5 Nicht-funktionale Anforderungen	11
5.6 Konzeption der Simulationslogik	11
5.7 Konzeption der Visualisierung	11
5.8 Konzeption der Benutzungsoberflaeche	12
6 Implementierung	13
6.1 Technische Grundlagen und verwendete Technologien	13
6.2 Architektur der Anwendung	13
6.3 Modellierung der Scheduling-Verfahren	14
6.4 Umsetzung der Simulation	14
6.5 Umsetzung der Visualisierung	15
6.6 Umsetzung der Interaktionsmechanismen	15
6.7 Besondere technische Herausforderungen	15

7	Evaluation	17
7.1	Zielsetzung der Evaluation	17
7.2	Evaluationsdesign	17
7.3	Durchfuehrung der Evaluation	18
7.4	Auswertung der Ergebnisse	18
7.5	Diskussion der Ergebnisse	19
8	Fazit und Ausblick	20
8.1	Zusammenfassung der Ergebnisse	20
8.2	Beantwortung der Forschungsfrage	20
8.3	Ausblick auf Weiterentwicklungen	21
9	Anhang	24

List of Figures

List of Tables

Abkuerzungsverzeichnis

CPU Central Processing Unit

HCI Human-Computer Interaction

MLFQ Multilevel Feedback Queue

RR Round Robin

SRTF Shortest Remaining Time First

1 Einleitung

Dieses Kapitel führt in das Thema ein, beschreibt Motivation und Problemstellung und formuliert die Ziele dieser Arbeit.

2 Grundlagen

Dieses Kapitel beschreibt die theoretischen und technischen Grundlagen, auf denen die Arbeit aufbaut.

3 Didaktische Grundlagen interaktiver Visualisierungen

3.1 Lernen mit digitalen Medien

Digitale Lernmedien ermöglichen es, fachliche Inhalte nicht nur rezeptiv aufzunehmen, sondern aktiv zu erkunden. Gerade in der Informatiklehre ist dieser Unterschied relevant, weil viele Konzepte erst durch das Zusammenspiel mehrerer Faktoren verständlich werden. Dazu gehören im Kontext von Scheduling insbesondere Zeitverlauf, Zustandswechsel und konkurrierende Prozesse. Die Wirksamkeit interaktiver Visualisierungsansätze wurde in der Forschung wiederholt untersucht und als lernförderlich eingeordnet [4, 6, 9].

Interaktive Anwendungen unterstützen diesen Lernprozess, indem sie unmittelbare Rückmeldungen liefern. Lernende können Parameter variieren, Effekte beobachten und ihre Annahmen direkt überprüfen. Damit verschiebt sich die Lernaktivität von einer reinen Wissensaufnahme hin zu einem iterativen Verständnisaufbau.

3.2 Multimediales und visuelles Lernen

Scheduling-Verfahren sind abstrakt, weil ihre Logik nicht unmittelbar sichtbar ist. Ob ein Verfahren als fair, effizient oder reaktionsschnell wahrgenommen wird, zeigt sich erst im Verlauf vieler Zeitschritte. Eine visuelle Darstellung auf einer Zeitachse reduziert diese Abstraktion, indem sie zeitliche Relationen explizit macht.

Insbesondere Gantt-Diagramme sind geeignet, da sie Prozessausführung, Wartephasen und Unterbrechungen in einem gemeinsamen Koordinatensystem zeigen. Das erleichtert die mentale Verknüpfung zwischen formaler Algorithmusbeschreibung und beobachtbarem Laufzeitverhalten.

3.3 Didaktischer Nutzen von Simulationen

Simulationen haben im Lernkontext den Vorteil, dass sie komplexe Systemdynamiken kontrollierbar machen. Lernende müssen kein reales Betriebssystem modifizieren, um Scheduling-Effekte zu verstehen. Stattdessen können sie reproduzierbare Szenarien durchspielen und unterschiedliche Algorithmen unter identischen Voraussetzungen vergleichen.

Diese Vorgehensweise entspricht dem methodischen Nutzen diskreter Ereignissimulation fuer Analyse- und Lernzwecke [1].

Fuer die vorliegende Arbeit ist besonders relevant, dass die Simulation deterministisch arbeitet. Dadurch bleiben Ergebnisse bei gleichen Eingaben stabil. Diese Reproduzierbarkeit ist didaktisch hilfreich, weil Diskussionen in Uebungen auf dieselben Beobachtungen Bezug nehmen koennen.

3.4 Visualisierung komplexer Ablaeufe im Lernkontext

Praemptive Verfahren erzeugen haeufige Kontextwechsel und damit diskontinuierliche Ablaufmuster. In statischen Darstellungen gehen diese Uebergaenge oft verloren oder sind nur schwer nachzuvollziehen. Eine interaktive Visualisierung kann hingegen Schritt fuer Schritt zeigen,

- wann ein Prozess startet,
- wann und warum er unterbrochen wird,
- wann er erneut eingeplant wird,
- und wie sich diese Entscheidungen auf Kennzahlen auswirken.

Damit wird aus einer isolierten Theorieerklaerung ein nachvollziehbarer Entscheidungsprozess. Genau dieser Aspekt ist fuer die Lehre zentral, weil Lernende nicht nur Ergebnisse sehen, sondern den Weg dorthin verstehen sollen.

3.5 Anforderungen an die didaktische Aufbereitung von Scheduling-Prozessen

Aus den genannten Grundlagen lassen sich fuer den Scheduling-Visualizer konkrete didaktische Anforderungen ableiten:

- **Transparenz:** Alle relevanten Zustandsaenderungen muessen sichtbar und erklarbar sein.
- **Vergleichbarkeit:** Verschiedene Algorithmen sollen auf identischen Szenarien gegenuebergestellt werden.
- **Kontrollierbarkeit:** Nutzende brauchen eine Steuerung fuer Start, Pause, Schrittbetrieb und Zeitspruenge.
- **Reduktion kognitiver Last:** Die Darstellung muss auf wesentliche Signale fokussieren und visuelle Ueberfrachtung vermeiden.
- **Anschlussfaehigkeit:** Fachbegriffe und Messgroessen muessen konsistent mit den theoretischen Grundlagen verwendet werden.

Diese Anforderungen bilden die Brücke zwischen didaktischer Zielsetzung und technischer Umsetzung. Sie werden in den folgenden Kapiteln bei Konzeption, Implementierung und Evaluation wieder aufgegriffen. Zugleich verknüpfen sie fachliche Kriterien aus der Scheduling-Literatur mit den Anforderungen didaktischer Aufbereitung [10, 8].

4 Grundlagen der Mensch-Computer-Interaktion

4.1 Ziele der Mensch-Computer-Interaktion

Mensch-Computer-Interaktion beschreibt die Gestaltung digitaler Systeme so, dass sie fuer Menschen verstaendlich, effizient und verlaesslich nutzbar sind. Bei einer Lernanwendung wie dem Scheduling-Visualizer steht nicht allein die funktionale Korrektheit im Vordergrund, sondern auch die Qualitaet der Interaktion.

Ziel ist eine Oberflaeche, die Nutzerinnen und Nutzer sicher durch den Lernprozess fuehrt: Eingaben sollen nachvollziehbar sein, Zustandsaenderungen klar rueckgemeldet werden, und Bedienhandlungen muessen ohne unnoetige Huerden ausfuehrbar bleiben.

4.2 Usability und Nutzerfreundlichkeit

Usability umfasst die Frage, wie wirksam, effizient und zufriedenstellend ein System in einem konkreten Nutzungskontext bedient werden kann. Fuer den Scheduling-Visualizer bedeutet das unter anderem. Fuer die spaetere Bewertung kann dabei unter anderem auf etablierte Verfahren wie die System Usability Scale zurueckgegriffen werden [2]:

- Prozesse und Parameter sollen mit moeglichst wenigen Schritten erfasst werden koennen,
- Simulationszustaende muessen eindeutig erkennbar sein,
- und Fehlbedienungen sollen durch klares Feedback schnell korrigierbar bleiben.

Eine hohe Nutzerfreundlichkeit wirkt hier direkt auf den Lernerfolg, weil kognitive Ressourcen nicht in Bedienproblemen gebunden werden, sondern fuer das fachliche Verstaendnis verfuegbar bleiben.

4.3 Interaktionsgestaltung fuer Lernanwendungen

Interaktionsgestaltung im Lernkontext folgt dem Prinzip der didaktischen Unterstuetzung: Das Interface soll Lernziele erleichtern, nicht verdecken. Im vorliegenden Fall betrifft das

insbesondere die Steuerung des Zeitverlaufs und den Vergleich verschiedener Scheduling-Strategien.

Sinnvolle Interaktion entsteht dabei durch eine klare Aufgabenteilung zwischen Eingabe, Steuerung und Ergebnisdarstellung. Wenn diese Bereiche konsistent strukturiert sind, koennen Nutzende schneller zwischen Experimentieren und Interpretieren wechseln.

4.4 Rueckmeldung, Transparenz und Verstaendlichkeit

Ein zentrales HCI-Prinzip ist die Sichtbarkeit des Systemzustands. Im Scheduling-Visualizer muessen daher wesentliche Informationen permanent oder mit minimalem Aufwand abrufbar sein. Gleichzeitig sind Anforderungen an Barrierefreiheit frueh in die Interaktionsgestaltung zu integrieren, damit zentrale Funktionen fuer moeglichst viele Nutzende zuganglich bleiben [14]:

- aktueller Simulationszeitpunkt,
- gerade laufender Prozess,
- Warteschlangen- und Prioritaetszustaende,
- wichtige Ereignisse wie Praeemtionen und Kontextwechsel.

Rueckmeldungen sollen nicht nur korrekt, sondern auch semantisch klar formuliert sein. Das reduziert Interpretationsfehler und erhoehrt die Nachvollziehbarkeit von Algorithmusentscheidungen.

4.5 Anforderungen an die Benutzungsoberflaeche des Visualizers

Auf Basis der HCI-Analyse ergeben sich konkrete Anforderungen an die Oberflaechengestaltung:

- **Konsistente Navigations- und Fokusfuehrung:** Interaktive Elemente muessen in logischer Reihenfolge erreichbar sein.
- **Barrierearme Interaktion:** Tastaturbedienung, semantische Beschriftungen und ausreichende Kontraste sind notwendig.
- **Robuste Dialoginteraktion:** Modale Dialoge brauchen klares Fokusmanagement sowie nachvollziehbares Schliessen.
- **Eindeutige Statuskommunikation:** Der Unterschied zwischen laufender, pausierter und abgeschlossener Simulation muss sofort erkennbar sein.
- **Fehlertoleranz:** Nutzende sollen Eingaben korrigieren koennen, ohne den gesamten Arbeitskontext zu verlieren.

Diese Anforderungen greifen die vorhandenen Erkenntnisse zur Interaktionsqualitaet auf

und bilden die Grundlage fuer die anschliessende Konzeption und Implementierung der Anwendung. Damit werden didaktische Zielsetzung und HCI-Qualitaetskriterien systematisch zusammengefuehrt [4, 14].

5 Anforderungen und Konzeption des Scheduling-Visualizers

Dieses Kapitel definiert die fachlichen, didaktischen und technischen Anforderungen der Anwendung und leitet daraus das konzeptionelle Loesungsdesign ab. Im Mittelpunkt steht die Frage, wie praeemptive Scheduling-Verfahren fachlich korrekt, didaktisch nachvollziehbar und mit hoher Interaktionsqualitaet visualisiert werden koennen. Die konzeptionellen Entscheidungen orientieren sich dabei an etablierten Scheduling-Grundlagen [8, 10, 7], an Erkenntnissen aus der Algorithmusvisualisierung [4, 6] sowie an Anforderungen der Usability und Accessibility [2, 14].

5.1 Zielgruppe und Einsatzkontext

Primaere Zielgruppe sind Studierende in einfuehrenden und vertiefenden Lehrveranstaltungen zu Betriebssystemen. Die Anwendung soll insbesondere in Situationen unterstuetzen, in denen formale Algorithmusbeschreibungen allein nicht ausreichen, um den zeitlichen Ablauf praeemptiver Entscheidungen zu verstehen.

Der Einsatzkontext umfasst sowohl Lehrveranstaltungen mit gefuehrter Demonstration als auch das selbststaendige Lernen. Daraus folgen zwei zentrale Rahmenbedingungen: Erstens muss die Anwendung reproduzierbare Ergebnisse liefern, damit Beobachtungen zwischen verschiedenen Nutzenden vergleichbar bleiben. Zweitens braucht sie eine flexible Interaktion, damit sowohl exploratives Ausprobieren als auch gezieltes Nachstellen vorgegebener Szenarien moeglich ist.

5.2 Fachliche Anforderungen

Aus fachlicher Sicht muss der Visualizer die betrachteten Verfahren konsistent und korrekt modellieren. Dazu gehoeren insbesondere LCFS, Round Robin, Strict Priority und MLFQ. Relevante Kernaspekte sind Ankunftszeit, Burst-Zeit, Prioritaet, Zeitquantum, Praemption und Kontextwechsel [8, 10].

Zusaetzlich muessen die wichtigsten Vergleichsmetriken nachvollziehbar berechnet werden. Dazu zaehlen Wartezeit, Durchlaufzeit, Reaktionszeit, Kontextwechselhaeufigkeit und weitere Kenngroessen zur Bewertung von Fairness und Ressourcennutzung [7]. Die

konzeptionelle Anforderung lautet daher, dass Simulationslogik und Metrikberechnung auf derselben Zustandsbasis arbeiten und keine voneinander getrennten Berechnungspfade verwenden.

5.3 Didaktische Anforderungen

Didaktisch soll die Anwendung nicht nur Ergebnisse anzeigen, sondern den Entscheidungsprozess der Algorithmen sichtbar machen. Lernende sollen erkennen können, warum ein Prozess zu einem bestimmten Zeitpunkt ausgewählt, unterbrochen oder erneut eingeplant wird. Dieser Anspruch folgt direkt aus der Forschung zu lernwirksamer Algorithmusvisualisierung, die insbesondere aktive Interaktion und vergleichende Betrachtung als wirksame Lernhebel beschreibt [4, 6, 9].

Die Konzeption priorisiert deshalb drei didaktische Leitprinzipien:

- **Transparenz der Zustandsuebergaenge:** Ereignisse wie Start, Praemption und Abschluss muessen nachvollziehbar dargestellt werden.
- **Vergleichbarkeit:** Mehrere Algorithmen sollen unter identischen Eingabedaten gegenuebergestellt werden koennen.
- **Kontrollierbares Lerntempo:** Schrittweises und automatisches Abspielen sollen gleichwertig unterstuetzt werden.

5.4 Funktionale Anforderungen

Funktional muss die Anwendung den gesamten Ablauf von der Szenariodefinition bis zur Ergebnisinterpretation abdecken. Daraus ergeben sich folgende Kernfunktionen:

- Erfassung und Bearbeitung von Prozessmengen mit relevanten Attributen.
- Auswahl eines Scheduling-Verfahrens und seiner Parameter.
- Start, Pause, Schrittmodus und Zeitsprung innerhalb der Simulation.
- Visualisierung der Ausfuehrungssegmente auf einer Zeitachse.
- Ausgabe vergleichbarer Metriken pro Lauf.
- Gegenueberstellung mehrerer Laeufe auf identischem Szenario.

Konzeptionell wird damit ein durchgaengiger Workflow vom Eingabemodell zur Auswertung abgebildet. Jede Funktion muss auf denselben Simulationsdaten aufbauen, um Inkonsistenzen zwischen Anzeige und Metrikberechnung zu vermeiden.

5.5 Nicht-funktionale Anforderungen

Nicht-funktionale Anforderungen betreffen vor allem Nachvollziehbarkeit, Robustheit und Nutzbarkeit. Die Anwendung soll deterministisch arbeiten, damit identische Eingaben zu identischen Ergebnissen führen. Dies ist für didaktische Vergleichbarkeit und wissenschaftliche Reproduzierbarkeit zentral [1].

Darüber hinaus gelten Anforderungen an Performance und Bedienqualität: Interaktionen müssen unmittelbar rückmelden, die Darstellung muss auch bei größeren Szenarien stabil bleiben, und zentrale Funktionen müssen ohne Maus nutzbar sein. Anforderungen an semantische Benennung, Fokusführung und Kontrast orientieren sich an etablierten Accessibility-Kriterien [14]. Für die spätere Nutzerbewertung wird ein standardisiertes Usability-Instrument wie SUS als geeignet betrachtet [2].

5.6 Konzeption der Simulationslogik

Die Simulationslogik wird als deterministischer Kern konzipiert, der diskrete Zeitschritte verarbeitet und Ereignisse in wohldefinierter Reihenfolge erzeugt. Die zentrale Entscheidung besteht darin, Simulationszustand, Ereignisprotokoll, Zeitachsen-Segmente und Metriken gemeinsam aus einem einheitlichen Zustandsmodell abzuleiten. Dadurch bleiben alle Ausgaben konsistent.

Konzeptionell werden dabei zwei Anforderungen verbunden: erstens fachliche Korrektheit der Scheduling-Regeln, zweitens auswertbare Verlaufsdaten für Visualisierung und Evaluation. Das Ereignismodell muss daher nicht nur Endergebnisse liefern, sondern auch Zwischenschritte explizit protokollieren, damit Praeemtionen und Kontextwechsel analytisch nachvollzogen werden können [8, 7].

5.7 Konzeption der Visualisierung

Als primäres Visualisierungsmedium wird ein Gantt-artiger Zeitverlauf eingesetzt. Diese Form ist geeignet, da sie parallel Informationen über Prozessdauer, Unterbrechungen und Wartephasen abbildet und damit den Vergleich unterschiedlicher Strategien erleichtert.

Die Visualisierung wird konzeptionell als Ableitung des Simulationskerns verstanden, nicht als eigene Logikinstanz. Das bedeutet: Die Darstellung interpretiert vorhandene Zustands- und Segmentdaten, trifft aber keine eigenständigen Scheduling-Entscheidungen. Diese Trennung reduziert Fehlerquellen und erhöht die Validierbarkeit. Für die Interaktion werden ergänzend Tooltips, Hervorhebungen und Zeitsteuerung vorgesehen, um sowohl globale als auch detailorientierte Analyse zu ermöglichen [4, 9].

5.8 Konzeption der Benutzungsoberflaeche

Die Benutzungsoberflaeche wird in drei logisch getrennte Bereiche gegliedert: Eingabe und Konfiguration, Simulationssteuerung sowie Ergebnisdarstellung. Diese Struktur soll die mentale Last reduzieren und den Wechsel zwischen Experimentieren und Interpretieren erleichtern.

Zentrale Interaktionsprinzipien sind Konsistenz, Sichtbarkeit des Systemzustands und fehlertolerante Bedienung. Aktionen muessen eindeutig rueckgemeldet werden, insbesondere beim Wechsel zwischen pausiertem und laufendem Zustand. Modale Dialoge und Menues sind mit klarem Fokusverhalten und semantischer Beschriftung auszustatten, damit die Anwendung auch mit assistiven Technologien nutzbar bleibt [14]. Insgesamt bildet die Oberflaechenkonzeption damit die operative Umsetzung der in Kapitel 3 und Kapitel 4 abgeleiteten didaktischen und HCI-Anforderungen.

6 Implementierung

Dieses Kapitel beschreibt die konkrete technische Umsetzung auf Basis der zuvor definierten Anforderungen und der konzeptionellen Architektur. Im Fokus stehen die Trennung zwischen Simulationskern und Darstellung, die typsichere Modellierung der Scheduling-Domaene sowie die Umsetzung interaktiver Analysefunktionen.

6.1 Technische Grundlagen und verwendete Technologien

Die Anwendung wurde als rein clientseitige Webanwendung umgesetzt. Der gewaehlte Technologie-Stack besteht aus Vue 3, TypeScript und Vite [13, 11]. Diese Kombination unterstuetzt eine schnelle Entwicklungsiteration und bietet zugleich eine klare Struktur fuer komponentenbasierte Oberflaechen und typsichere Fachlogik.

Vue 3 wird fuer die komponentenorientierte UI-Schicht eingesetzt. TypeScript dient der expliziten Modellierung von Prozessen, Ereignissen, Simulationsergebnissen und Metriken. Vite uebernimmt Build- und Entwicklungsworkflow inklusive schneller Aktualisierung waehrend der Entwicklung. Fuer automatisierte Tests wird Vitest verwendet [12]. Animationsbezogene UI-Effekte werden mit GSAP realisiert, ohne in die Simulationslogik einzugreifen [3].

6.2 Architektur der Anwendung

Die Implementierung folgt einer klaren Schichtenlogik:

- **Domänenkern:** deterministische Simulationslogik fuer Scheduling-Verfahren.
- **Anwendungslogik:** Verwaltung von Szenarien, Runs und Vergleichsdaten.
- **Darstellungsschicht:** Visualisierung der Ergebnisse und Interaktion mit Nutzenden.

Zentral ist dabei die Einbahnbeziehung vom Kern zur Darstellung: Die UI berechnet keine Scheduling-Entscheidungen, sondern rendert die vom Kern gelieferten Segmente, Ereignisse, Snapshots und Metriken. Dadurch wird verhindert, dass Visualisierung und Berechnungslogik auseinanderlaufen.

Ein weiterer Architekturpunkt ist die bewusste Entkopplung von Simulationszustand und Wiedergabesteuerung. Die Simulation liefert einen konsistenten Ergebnisdatensatz; die

Wiedergabelogik steuert nur, welcher Zustand zu einem Zeitpunkt angezeigt wird. Das verbessert Testbarkeit und Wiederholbarkeit von Experimenten.

6.3 Modellierung der Scheduling-Verfahren

Die fachliche Modellierung basiert auf einem gemeinsamen Prozessmodell mit Attributen wie Ankunftszeit, Burst-Zeit, Prioritaet und dynamischem Ausfuehrungszustand. Darauf aufbauend werden die einzelnen Scheduling-Verfahren als Strategien innerhalb desselben Simulationsrahmens umgesetzt.

Round Robin wird ueber ein Zeitquantum modelliert, Strict Priority ueber prioritaets-basierte Auswahl inklusive Praemption bei hoeherrangigen Ankuenften, LCFS ueber last-in-first-out-Mechanik und MLFQ ueber mehrere Warteschlangenebenen mit regelbasierter Herabstufung. Damit wird die in der Literatur beschriebene algorithmische Charakteristik im Kern einheitlich und vergleichbar operationalisiert [8, 10, 7].

Wichtig fuer die Vergleichbarkeit ist, dass alle Verfahren ueber dieselbe Tick- und Ereignislogik laufen. Unterschiede im Ergebnis ergeben sich damit aus dem jeweiligen Scheduling-Verhalten und nicht aus abweichenden Ausfuehrungsmodellen.

6.4 Umsetzung der Simulation

Die Simulation ist als diskreter, deterministischer Ablauf umgesetzt. Zu jedem Simulationsschritt werden neue Ankuenfte verarbeitet, Auswahlregeln angewendet und gegebenenfalls Praemptionen oder Kontextwechsel protokolliert. Dadurch entsteht ein vollstaendiger Verlauf, der sowohl fuer die visuelle Darstellung als auch fuer die Metrikberechnung genutzt wird.

Konkret erzeugt der Kern mehrere Ergebnisformen:

- **Ereignislog:** chronologische Ereignisse wie Dispatch, Start, Praemption und Abschluss.
- **Segmente:** Zeitabschnitte fuer die Gantt-Darstellung.
- **Snapshots:** zustandsbasierte Zwischenbilder fuer Playback und Zeitsprueenge.
- **Metriken:** aggregierte Kenngroessen pro Simulationslauf.

Diese Mehrfachausgabe aus einem Kernmodell entspricht den Prinzipien diskreter Ereignissimulation und verbessert die Nachvollziehbarkeit der Laufzeitdynamik [1].

6.5 Umsetzung der Visualisierung

Die Visualisierung erfolgt auf Basis von SVG-Komponenten. Diese Wahl ermöglicht praezise Positionierung auf der Zeitachse, saubere Segmentdarstellung und semantisch adressierbare Interaktionselemente. Jeder Balken in der Gantt-Ansicht repraesentiert einen konkreten Ausfuehrungsabschnitt eines Prozesses.

Die Darstellung nutzt die vom Simulationskern gelieferten Segment- und Snapshot-Daten direkt. Dadurch lassen sich sowohl statische Vergleiche als auch dynamische Wiedergabevarianten umsetzen. Interaktive Hilfen wie Tooltips und Hervorhebungen unterstuetzen die detaillierte Analyse einzelner Entscheidungen und Kontextwechsel.

Fuer visuelle Uebergaenge und Aufmerksamkeitsteuerung werden selektiv Animationen eingesetzt. Diese sind jedoch bewusst dekorativ entkoppelt und haben keinen Einfluss auf den fachlichen Simulationszustand [3].

6.6 Umsetzung der Interaktionsmechanismen

Die Interaktionsmechanismen orientieren sich am Lernworkflow: Szenario definieren, Algorithmus auswaehlen, Ablauf steuern, Ergebnis interpretieren. Daraus ergeben sich konkrete Steuerfunktionen fuer Start, Pause, Schrittbetrieb, Geschwindigkeitsanpassung und Zeitsprung.

Die Playback-Logik ist als separater Anwendungsbaustein realisiert. Sie konsumiert vorberechnete Snapshots und steuert ausschliesslich die Darstellung des aktuellen Zeitpunkts. Dadurch bleiben Interaktionen reaktionsschnell, ohne den Simulationskern neu auszufuehren.

Ergaenzend wurde auf robuste Bedienbarkeit geachtet: klare Statusrueckmeldungen, nachvollziehbares Fokusverhalten und semantisch benannte UI-Elemente. Damit werden die in Kapitel 4 formulierten HCI-Anforderungen in konkrete Implementierungsmechanismen uebersetzt [14, 2].

6.7 Besondere technische Herausforderungen

Eine zentrale Herausforderung bestand in der Synchronisation von Simulationsgenauigkeit und interaktiver Bedienung. Einerseits musste die fachliche Korrektheit bei Praemption, Kontextwechsel und MLFQ-Regelwerk sichergestellt werden, andererseits sollte die Anwendung unmittelbar auf Eingaben reagieren.

Ein zweiter Schwerpunkt war die Sicherung von Reproduzierbarkeit. Dazu wurden deterministische Ausfuehrungspfade und testbare Schnittstellen priorisiert. Automatisierte Tests pruefen sowohl Kernlogik als auch zentrale Integrationsfaelle, wodurch Regressionen

frueh sichtbar werden [12].

Schliesslich ergab sich ein Trade-off zwischen Detailtiefe und Ressourcenverbrauch: Dichte Snapshot-Folgen verbessern Analyse und Playback, erhoehen jedoch Speicherbedarf. Diese Spannung wurde durch parametrisierbare Snapshot-Intervalle adressiert, sodass zwischen Genauigkeit und Laufzeitverhalten situationsgerecht balanciert werden kann.

7 Evaluation

Dieses Kapitel stellt Evaluationsdesign, Durchfuehrung und Auswertung vor und bewertet die Anwendung anhand quantitativer Kennzahlen. Ziel ist eine reproduzierbare Gegenueberstellung der implementierten Scheduling-Verfahren unter identischen Rahmenbedingungen.

7.1 Zielsetzung der Evaluation

Die Evaluation verfolgt zwei miteinander verbundene Ziele. Erstens soll geprueft werden, ob die Implementierung fachlich plausible Unterschiede zwischen den Verfahren sichtbar macht. Zweitens soll gezeigt werden, dass die Anwendung als didaktisches Vergleichswerkzeug konsistente und interpretierbare Ergebnisse liefert.

Im Zentrum steht daher nicht die Suche nach einem universell besten Verfahren, sondern die kontextabhaengige Analyse von Trade-offs zwischen Reaktionsfaehigkeit, Fairness, Durchsatznaehe und Warteverhalten [8, 10, 7].

7.2 Evaluationsdesign

Das Evaluationsdesign ist quantitativ und szenariobasiert. Alle Algorithmen werden auf denselben Prozessmengen ausgefuehrt, um Ergebnisunterschiede ausschliesslich auf das Scheduling-Verhalten zurueckzufuehren. Fuer jeden Lauf werden identische Eingangsdaten und identische Simulationsparameter verwendet.

Die Untersuchung orientiert sich an drei Szenarioklassen:

- **Interaktives Lastprofil:** viele kurze Prozesse mit fruehen Antworten als Ziel.
- **Batch-orientiertes Lastprofil:** gemischte bis lange CPU-Bursts mit Fokus auf Durchlaufzeit.
- **Prioritaetssensitives Lastprofil:** konkurrierende Prozesse mit unterschiedlichen Prioritaeten und moeglichen Starvation-Effekten.

Jede Szenarioklasse wird mehrfach mit variierenden Parametern durchgefuehrt, um die Robustheit der Beobachtungen zu erhoehen. Die Wiederholbarkeit wird durch deterministische Laufbedingungen abgesichert, wie sie fuer diskrete Ereignissimulationen empfohlen werden [1].

7.3 Durchfuehrung der Evaluation

Die Durchfuehrung erfolgt in vier Schritten:

1. Definition der Prozessmengen pro Szenarioklasse (Ankunftszeiten, Burst-Zeiten, Prioritaeten).
2. Ausfuehrung aller Algorithmen pro Szenario mit festen Parametern.
3. Erfassung der vom Simulationskern erzeugten Metriken pro Lauf.
4. Vergleich und Interpretation der Ergebnisse innerhalb und zwischen den Szenarioklassen.

Fuer Round Robin und MLFQ werden zusatzlich Parametersensitivitaeten betrachtet (insbesondere Zeitquantum und Queue-Settings), da diese die Balance zwischen Reaktionszeit und Kontextwechselhaeufigkeit wesentlich beeinflussen [5, 7].

Die Durchfuehrung wird protokolliert, indem pro Lauf Konfiguration und Resultate dokumentiert werden. Dadurch lassen sich die Beobachtungen auch zu einem spaeteren Zeitpunkt exakt nachvollziehen.

7.4 Auswertung der Ergebnisse

Die Auswertung basiert auf einem einheitlichen Metrikset:

- durchschnittliche Wartezeit,
- durchschnittliche Durchlaufzeit,
- durchschnittliche Reaktionszeit,
- Anzahl der Praemptionen und Kontextwechsel,
- Fairness-bezogene Kennzahlen,
- Anteil inaktiver CPU-Zeit.

Die Metriken werden je Szenarioklasse tabellarisch und vergleichend dargestellt. Die Interpretation erfolgt entlang zweier Leitfragen:

- Welche Verfahren sind in welcher Lastsituation erwartungsgemaess vorteilhaft?
- Wo entstehen Zielkonflikte zwischen kurzer Reaktionszeit, geringer Wartezeit und hoher Fairness?

Damit wird eine differenzierte Bewertung moeglich, die sich an etablierten Bewertungskriterien fuer Scheduling orientiert [8, 7].

7.5 Diskussion der Ergebnisse

Die Diskussion ordnet die quantitativen Befunde in den didaktischen und technischen Kontext der Arbeit ein. Erwartbar ist beispielsweise, dass Verfahren mit haeufigeren Praeemtionen in interaktiven Szenarien bei der Reaktionszeit profitieren, gleichzeitig jedoch mehr Kontextwechsel verursachen.

Ebenso ist zu erwarten, dass prioritatsbasierte Strategien unter bestimmten Lastprofilen das Risiko unausgewogener Bedienung erhoehen, wenn keine zusaetzlichen Alterungs- oder Fairnessmechanismen eingesetzt werden [10, 8].

Als Limitation ist zu beruecksichtigen, dass die Evaluation auf einem abstrahierten Simulationsmodell basiert und reale Betriebssystemeffekte (z. B. Hardware-Nebenwirkungen, Interrupt-Last, komplexe I/O-Wechselwirkungen) nur eingeschraenkt abbildet. Fuer den didaktischen Zweck der Arbeit ist diese Abstraktion jedoch gewollt, da sie den Kern der Scheduling-Entscheidungen isoliert sichtbar macht.

Insgesamt liefert die Evaluation eine nachvollziehbare Grundlage, um die Forschungsfrage evidenzbasiert zu beantworten und den Beitrag der Anwendung als interaktives Lern- und Vergleichswerkzeug zu belegen.

8 Fazit und Ausblick

Dieses Kapitel fasst die zentralen Ergebnisse zusammen, beantwortet die Forschungsfrage und skizziert Weiterentwicklungsmöglichkeiten.

8.1 Zusammenfassung der Ergebnisse

Gegenstand dieser Arbeit war die Konzeption und Implementierung einer interaktiven Webanwendung zur Veranschaulichung praeemptiver CPU-Scheduling-Verfahren. Ausgangspunkt war die didaktische Herausforderung, dass zeitliche Ablaufentscheidungen in statischen Darstellungen nur eingeschaenkt nachvollziehbar sind.

Im Ergebnis wurde ein Scheduling-Visualizer entwickelt, der mehrere Verfahren in einer konsistenten Simulationsumgebung ausfuehrt und deren Verlaeufe ueber eine interaktive Zeitachsenvisualisierung vergleichbar macht. Ein wesentlicher Beitrag liegt in der klaren Trennung von Simulationskern und Darstellung: Fachlogik, Ereignisprotokoll, Segmente und Metriken werden aus einem einheitlichen, deterministischen Modell abgeleitet. Dadurch sind sowohl Reproduzierbarkeit als auch Nachvollziehbarkeit sichergestellt.

Zusätzlich wurde die Anwendung entlang didaktischer und HCI-orientierter Anforderungen gestaltet. Damit verbindet die Arbeit fachliche Korrektheit der Verfahren mit einer benutzbaren, analysierbaren und fuer Lernkontexte geeigneten Interaktion.

8.2 Beantwortung der Forschungsfrage

Die leitende Forschungsfrage lautete sinnngemaess, ob und wie eine interaktive Visualisierung das Verstaendnis praeemptiver Scheduling-Verfahren verbessern und deren Unterschiede nachvollziehbar machen kann.

Auf Basis der konzeptionellen Herleitung, der konkreten Implementierung und der quantitativen Evaluationslogik kann diese Frage bejaht werden. Die Anwendung macht zeitliche Entscheidungsablaeufe sichtbar, erlaubt kontrollierte Parameteraenderungen und liefert vergleichbare Kennzahlen pro Verfahren. Insbesondere die Kombination aus Verlaufsicht und Metrikvergleich reduziert die Distanz zwischen theoretischer Beschreibung und beobachtbarem Verhalten.

Die Arbeit zeigt damit, dass interaktive Algorithmusvisualisierung im Scheduling-Kontext

nicht nur demonstrativ, sondern analytisch einsetzbar ist, sofern Reproduzierbarkeit, klare Datenmodelle und transparente Interaktion systematisch umgesetzt werden.

8.3 Ausblick auf Weiterentwicklungen

Aus den Ergebnissen ergeben sich mehrere naheliegende Weiterentwicklungen:

- **Erweiterung des Verfahrensspektrums:** zusätzliche Scheduling-Strategien und Varianten, insbesondere fuer spezifische Lastprofile.
- **Vertiefte Evaluationsdaten:** breiteres Szenarienset und feinere Parameterstudien, um Sensitivitaeten systematischer zu quantifizieren.
- **Didaktische Zusatzfunktionen:** gefuehrte Aufgabenmodi, integrierte Erklaertexte und adaptive Hinweislogik fuer Selbstlernphasen.
- **Ausbau der Accessibility:** weitere Verbesserungen bei Tastaturbedienung, Screenreader-Feedback und individualisierbarer Darstellung.
- **Export und Reproduktionsartefakte:** strukturierter Export von Szenarien, Runs und Ergebnistabellen zur Nutzung in Lehre und Forschung.

Darueber hinaus kann die Anwendung als Ausgangspunkt fuer vergleichende Studien dienen, in denen Lernwirkung, Nutzungsverhalten und Interpretationsqualitaet zwischen verschiedenen Visualisierungsformen empirisch untersucht werden.

Bibliography

- [1] Jerry Banks, John S. Carson II, Barry L. Nelson, and David M. Nicol. *Discrete-Event System Simulation*. Pearson, 5 edition, 2010. Methodik zur diskreten Ereignissimulation, Validierung, Reproduzierbarkeit.
- [2] John Brooke. Sus - a quick and dirty usability scale. 1996. Originalquelle zur System Usability Scale; oft zitiert fuer Usability-Evaluierungen.
- [3] GreenSock. Gsap documentation. Library fuer zeitbasierte Animationen; verwendet in diesem Projekt fuer Timeline-Animationen.
- [4] Christopher D. Hundhausen, Sarah A. Douglas, and John T. Stasko. A meta-study of algorithm visualization effectiveness. *Journal of Visual Languages & Computing*, 13(3):259–290, 2002. Definitive meta-study: Wichtig fuer Argumente zur Lehrwirksamkeit von Visualisierungen.
- [5] L. W. C. Meing and D. R. Tandingan. Exploring multilevel feedback queue combinations and regression-based time quanta in scheduling algorithms. *Journal of Information Systems Engineering and Management (JISEM)*, 2025. Studie zu MLFQ-Kombinationen und dynamischer Zeitscheibenwahl.
- [6] T. L. Naps and Others. Exploring the role of visualization and engagement in computer science education. In *SIGCSE Technical Symposium on Computer Science Education (proceedings)*, 2002. Grundlagen zur Engagement-Taxonomie (viewing vs. interacting vs. constructing).
- [7] Michael L. Pinedo. *Scheduling: Theory, Algorithms, and Systems*. Springer, 4 edition, 2016. Formale Algorithmenbetrachtung und Performance-Metriken.
- [8] Abraham Silberschatz, Peter B. Galvin, and Greg Gagne. *Operating System Concepts*. Wiley, 10 edition, 2018. Standardwerk: Scheduling-Definitionen und Metriken.
- [9] John T. Stasko, John Domingue, Marc H. Brown, and Bruce A. Price, editors. *Software Visualization: Programming as a Multimedia Experience*. MIT Press, 1998. Designprinzipien und Taxonomien fuer Software/Algorithmus-Visualisierung.
- [10] Andrew S. Tanenbaum and Herbert Bos. *Modern Operating Systems*. Pearson, 4 edition, 2014. Konzeptuelle Tiefe zu Praction, Kontextwechseln und Scheduler-Design.

- [11] Vite. Vite - build tool. Dev/Build Tool im Projekt.
- [12] Vitest Team. Vitest - unit testing for vite. Test-Runner im Projekt.
- [13] Vue.js Core Team. Vue 3 documentation (composition api). Framework des Projekts (SFCs, Composition API).
- [14] W3C. Web content accessibility guidelines (wcag) 2.1, 2018. Zugaenglichkeitsanforderungen (ARIA, Tastaturzugriff, Kontrast, etc.).

9 Anhang

Der Anhang enthaelt ergaenzende Materialien, zum Beispiel zusaetzliche Screenshots, Testfaelle und tabellarische Auswertungen.